

Hadoop WordCount Program (Eclipse + Ubuntu Terminal)

Step 1 — Write the Program in Eclipse

1. Open **Eclipse**
2. Create a **Java Project**
3. Create a Java class file:

```
WordCount.java
```

4. Paste the Hadoop WordCount code inside it.

Eclipse is only used to **type the code**, not to run Hadoop.

Step 2 — Copy the File into a Folder

After writing the program:

1. Right click `WordCount.java` → Copy
2. Create a folder in Ubuntu/Linux:

Example:

```
hadoop_lab
```

3. Paste the file into that folder.

Now your folder looks like:

```
hadoop_lab/  
  WordCount.java
```

Step 3 — Create Input File

Inside the same folder, create a file called:

```
input.txt
```

Example content:

```
hello hadoop  
hello world  
hello mapreduce
```

Now folder becomes:

```
hadoop_lab/  
  WordCount.java  
  input.txt
```

Step 4 — Open Terminal

Terminal is where we type commands instead of clicking.

Open Terminal:

Press:

```
Ctrl + Alt + T
```

Terminal looks like:

```
user@ubuntu:~$
```

Step 5 — Basic Linux Commands (Must Know)

Command	Meaning
<code>pwd</code>	Shows current folder
<code>ls</code>	Shows files inside folder
<code>cd folder</code>	Enter folder
<code>cd ..</code>	Go back

Example:

```
ls
```

Shows files.

Step 6 — Go Inside Program Folder

If your folder is `hadoop_lab` :

```
cd hadoop_lab
```

Check files:

```
ls
```

You should see:

```
WordCount.java  
input.txt
```

Step 7 — Start Hadoop

Start all Hadoop services:

```
start-all.sh
```

This starts:

- NameNode
 - DataNode
 - ResourceManager
 - NodeManager
-

Step 8 — Format NameNode (Only First Time)

If Hadoop is running for the first time, you must do:

```
stop-all.sh  
hdfs namenode -format
```

-> Format is done only once

-> Never repeat format again or HDFS data will be lost

After format, start Hadoop again:

```
start-all.sh
```

Step 9 — Check Hadoop is Running

Run:

```
jps
```

You must see:

```
NameNode  
DataNode  
SecondaryNameNode  
ResourceManager  
NodeManager
```

Hadoop is running correctly.

Step 10 — Create Input Directory in HDFS

Hadoop jobs read input only from **HDFS**, not local files.

Create input folder:

```
hdfs dfs -mkdir -p /wordcount/input
```

Meaning:

- `mkdir` → make folder
 - `-p` → creates parent folders automatically
-

Step 11 — Upload input.txt into HDFS

```
hdfs dfs -put input.txt /wordcount/input
```

Check upload:

```
hdfs dfs -ls /wordcount/input
```

Output:

```
input.txt
```

Input is now inside Hadoop storage (HDFS).

Step 12 — Compile the Java Program

Hadoop cannot run `.java` directly.

We must compile first.

```
javac -classpath $(hadoop classpath) WordCount.java
```

After compiling, check:

```
ls
```

You will see `.class` files like:

```
WordCount.class  
WordCount$TokenizerMapper.class  
WordCount$IntSumReducer.class
```

Step 13 — Create JAR File

Hadoop runs jobs using `.jar`.

```
jar cf wordcount.jar WordCount*.class
```

Now jar is created:

```
wordcount.jar
```

Step 14 — Remove Old Output Folder (If Rerunning)

Hadoop never overwrites output folders.

So delete previous output:

```
hdfs dfs -rm -r /wordcount/output
```

(Only needed if output exists.)

Step 15 — Run WordCount Job

Run the MapReduce job:

```
hadoop jar wordcount.jar WordCount /wordcount/input /wordcount/output
```

Explanation:

Part	Meaning
<code>hadoop jar</code>	Run Hadoop program
<code>wordcount.jar</code>	Jar file
<code>WordCount</code>	Driver class
<code>/wordcount/input</code>	Input folder in HDFS
<code>/wordcount/output</code>	Output folder in HDFS

Hadoop runs Mapper + Reducer now.

Step 16 — View Output

Final result is stored in HDFS:

```
hdfs dfs -cat /wordcount/output/part-r-00000
```

Example output:

```
hello      3
hadoop     1
mapreduce  1
world      1
```

WordCount executed successfully.

Step 17 — Stop Hadoop Services (Very Important)

After finishing lab, stop Hadoop:

```
stop-all.sh
```

This frees RAM and background processes.

Full Command List

```
cd hadoop_lab
ls

hdfs namenode -format      # Only first time
start-all.sh

jps

hdfs dfs -mkdir -p /wordcount/input
```

```
hdfs dfs -put input.txt /wordcount/input
```

```
javac --release 11 -classpath $(hadoop classpath) WordCount.java  
jar cf wordcount.jar WordCount*.class
```

```
hdfs dfs -rm -r /wordcount/output
```

```
hadoop jar wordcount.jar WordCount /wordcount/input /wordcount/output
```

```
hdfs dfs -cat /wordcount/output/part-r-00000
```

```
stop-all.sh
```
